

INPT

Institut National des Postes et Télécommunications

DÉTECTION DE FRAUDE BANCAIRE

*Analyse Comparative des Algorithmes
d'Apprentissage Automatique*

Réalisé par :

YOUSSEF EL KROUK

Filière : Science de Données

Année universitaire : **2025-2026**

Sujet : Évaluation comparative de cinq algorithmes d'apprentissage automatique (Random Forest, AdaBoost, CatBoost, XGBoost, LightGBM) pour la détection de fraude par carte bancaire sur un dataset de 284,807 transactions.

Résultat Principal :

LightGBM : **ROC-AUC = 0.976**

Table des matières

1. Introduction	4
1.1 Contexte et Motivation	4
1.2 Importance du Machine Learning	4
1.3 Objectifs du Rapport	5
1.4 Organisation du Rapport	5
2. Description du Dataset	6
2.1 Source et Contexte	6
2.2 Caractéristiques Principales	6
2.2.1 Taille et Distribution	6
2.2.2 Caractéristiques des Features	7
2.3 Qualité des Données	8
2.3.1 Absence de Valeurs Manquantes	8
2.3.2 Normalisation des Features	8
3. Méthodologie et Préparation des Données	9
3.1 Prétraitement des Données	9
3.1.1 Normalisation et Scaling	9
3.1.2 Gestion du Déséquilibre	9
3.2 Stratégie de Partitionnement	9
3.2.1 Division Train/Validation/Test	9
3.3 Métriques d'Évaluation	10
3.3.1 Pourquoi Pas Accuracy ?	10
3.3.2 ROC-AUC (Area Under Curve)	10
3.3.3 Précision et Rappel	10
3.3.4 F1-Score	10
4. Random Forest - Théorie et Implémentation	11
4.1 Concepts Fondamentaux	11
4.1.1 Arbres de Décision	11
4.2 Ensemble Learning et Bagging	11
4.2.1 Motivation de l'Ensemble	11
4.3 Random Forest	12
4.3.1 Architecture et Algorithme	12

4.3.2	Importance des Features dans Random Forest	12
4.4	Application à la Détection de Fraude	12
4.4.1	Hyperparamètres Configurés	12
4.4.2	Performance Observée	13
5.	AdaBoost - Théorie et Implémentation	14
5.1	Principes du Boosting	14
5.1.1	Boosting vs Bagging	14
5.2	AdaBoost (Adaptive Boosting)	14
5.2.1	Algorithme Détaillé	14
5.2.2	Interprétation Mathématique	15
5.3	Application à la Détection de Fraude	15
5.3.1	Avantages pour notre Problème	15
5.3.2	Configuration et Performance	15
6.	Gradient Boosting et XGBoost	16
6.1	Théorie du Gradient Boosting	16
6.1.1	Perspective de l'Optimisation	16
6.1.2	Algorithme Gradient Boosting Générique	16
6.2	XGBoost (Extreme Gradient Boosting)	17
6.2.1	Innovations de XGBoost	17
6.2.2	Avantages de XGBoost	17
6.3	Configuration et Application	17
6.3.1	Hyperparamètres XGBoost	17
7.	CatBoost - Théorie et Implémentation	18
7.1	Spécificités de CatBoost	18
7.1.1	Motivation pour les Features Catégorielles	18
7.2	Ordered Boosting	18
7.2.1	Problème du Boosting Traditionnel	18
7.3	Arbres Obélisques	19
7.3.1	Définition et Avantages	19
7.4	Configuration CatBoost pour Notre Problème	19
7.4.1	Hyperparamètres	19
8.	LightGBM - Théorie et Implémentation	20
8.1	Innovation Clé : Leaf-wise Tree Growth	20
8.1.1	Level-wise vs Leaf-wise	20
8.1.2	Comparaison Mathématique	20
8.2	Gradient-based One-Side Sampling (GOSS)	20
8.2.1	Réduction de Données Sans Perte	20
8.3	Histogram-based Learning	21
8.3.1	Groupe des Features Continues	21

8.4	Configuration et Performance	21
8.4.1	Hyperparamètres Optimisés	21
8.4.2	Justification du Succès	21
9.	Résultats et Comparaisons Expérimentales	22
9.1	Performance Globale	22
9.1.1	ROC-AUC Scores	22
9.1.2	Métriques Détaillées	22
9.2	Analyse de la Généralisabilité	22
9.2.1	Écart Train-Test	22
9.3	Importance des Features	23
9.3.1	Top Features selon LightGBM	23
10.	Conclusion et Perspectives Futures	24
10.1	Résumé des Findings	24
10.2	Recommandations Pratiques	24
10.2.1	Pour le Déploiement en Production	24
10.3	Perspectives Futures	25
10.3.1	Améliorations Possibles	25
10.4	Impact Potentiel	25
10.5	Conclusion Finale	25

Chapitre 1

Introduction

1.1 Contexte et Motivation

La détection de fraude par carte bancaire est l'un des défis majeurs de l'industrie financière mondiale. Chaque année, des milliards de dollars sont perdus à cause de transactions frauduleuses, affectant à la fois les consommateurs et les institutions financières. Selon les rapports récents, environ 1 transaction sur 500 est frauduleuse, ce qui crée un déséquilibre extrême entre les classes de données.

Le problème fondamental en détection de fraude bancaire réside dans l'extrême déséquilibre des données. Les transactions légitimes représentent environ 99.83% des cas, tandis que les fraudes ne constituent que 0.17% du dataset. Ce déséquilibre rend les algorithmes traditionnels inefficaces, car ils tendent à optimiser pour la classe majoritaire, ignorant les signaux de fraude.

1.2 Importance du Machine Learning

Les méthodes traditionnelles basées sur des règles experts montrent leurs limites face à l'évolution constante des techniques de fraude. Les fraudeurs s'adaptent continuellement, contournant les systèmes de détection existants. L'apprentissage automatique offre une solution flexible et adaptable qui peut :

- **Apprendre des modèles complexes** : Identifier des patterns non-linéaires et des interactions feature subtiles
- **S'adapter continuellement** : Être ré-entraîné régulièrement avec de nouvelles données
- **Généraliser** : Détecter des fraudes jamais observées dans les données d'entraînement
- **Scalabilité** : Traiter des millions de transactions en temps réel

1.3 Objectifs du Rapport

Ce rapport poursuit les objectifs suivants :

1. Présenter une analyse théorique complète de chaque algorithme utilisé
2. Évaluer la performance de cinq algorithmes majeurs sur le problème de détection de fraude
3. Comparer les résultats selon plusieurs métriques appropriées pour données déséquilibrées
4. Analyser la généralisabilité et les risques de surapprentissage
5. Fournir des recommandations pour une implémentation en production

1.4 Organisation du Rapport

Le rapport est organisé en 10 chapitres :

- **Chapitre 2** : Description du dataset et exploration exploratoire
- **Chapitre 3** : Méthodologie et préparation des données
- **Chapitre 4-8** : Théorie et implémentation de chaque algorithme
- **Chapitre 9** : Résultats expérimentaux et comparaisons
- **Chapitre 10** : Discussion, conclusions et perspectives futures

Chapitre 2

Description du Dataset

2.1 Source et Contexte

Le dataset utilisé est le **Credit Card Fraud Detection** disponible sur Kaggle, compilé par le *Machine Learning Group* de l'Université Libre de Bruxelles (ULB). Les données concernent des transactions bancaires effectuées par des clients européens en septembre 2013.

2.2 Caractéristiques Principales

2.2.1 Taille et Distribution

TABLE 2.1 – Statistiques du Dataset

Attribut	Valeur
Nombre total de transactions	284,807
Transactions légitimes	284,315 (99.827%)
Transactions frauduleuses	492 (0.173%)
Ratio de déséquilibre	1 :579
Nombre de features	31
Nombre de composantes PCA	28

Ce déséquilibre extrême est représentatif des problèmes réels de détection de fraude. Une majorité écrasante de transactions légales crée un défi méthodologique majeur.

2.2.2 Caractéristiques des Features

Features PCA-Transformées (V1-V28)

Due aux contraintes de confidentialité, les 28 features originales ont été transformées via Analyse en Composantes Principales (PCA). Ces composantes :

- Sont orthogonales entre elles (non corrélées)
- Sont déjà standardisées (moyenne 0, écart-type 1)
- Capturent les variations significatives dans les patterns de transaction
- Protègent l'anonymat des données originales

Feature Temps

La feature **Time** enregistre le nombre de secondes écoulées depuis la première transaction du dataset :

- **Plage** : 0 à 172,792 secondes (environ 48 heures)
- **Moyenne** : 94,813.86 secondes
- **Écart-type** : 47,488.15 secondes
- **Interprétation** : Capture les patterns temporels possibles dans la fraude

Feature Montant

Le **Amount** représente le montant de la transaction en euros :

- **Plage** : 0 à 25,691.16 EUR
- **Moyenne** : 88.35 EUR
- **Médiane** : 22.00 EUR
- **Distribution** : Fortement asymétrique à droite
- **Utilisation** : Peut identifier les transactions de montant inusuel

Variable Cible (Class)

$$\text{Class} = \begin{cases} 0 & \text{si transaction légitime} \\ 1 & \text{si transaction frauduleuse} \end{cases} \quad (2.1)$$

2.3 Qualité des Données

2.3.1 Absence de Valeurs Manquantes

Une analyse complète révèle :

- Aucune valeur manquante (NaN) dans les 284,807 transactions
- Aucun doublon exact détecté
- Tous les types de données sont correctement formatés
- Cohérence temporelle : les transactions s'étendent sur 48 heures consécutives

2.3.2 Normalisation des Features

Les features PCA sont déjà standardisées, mais Time et Amount nécessitent une normalisation pour assurer une échelle commune lors de l'entraînement.

Chapitre 3

Méthodologie et Préparation des Données

3.1 Prétraitement des Données

3.1.1 Normalisation et Scaling

$$x_{\text{normalisé}} = \frac{x - \mu}{\sigma} \quad (3.1)$$

où μ est la moyenne et σ l'écart-type. Cette transformation assure que toutes les features contribuent équitablement à l'apprentissage du modèle.

3.1.2 Gestion du Déséquilibre

- **Stratified Split** : Maintenir la distribution des classes dans les ensembles de train/validation/test
- **Métriques Appropriées** : Utiliser ROC-AUC plutôt que l'accuracy
- **Boosting Inhérent** : Les algorithmes de boosting pondèrent naturellement les erreurs de classification
- **Cross-Validation Stratifiée** : K-fold avec équilibrage des classes dans chaque fold

3.2 Stratégie de Partitionnement

3.2.1 Division Train/Validation/Test

Le partitionnement est effectué de manière **stratifiée**, préservant le ratio de fraude (0.173%) dans chaque ensemble. Cette approche est critique pour éviter que la classe minoritaire soit trop représentée ou sous-représentée dans un ensemble quelconque.

3.3 Métriques d'Évaluation

3.3.1 Pourquoi Pas Accuracy ?

$$\text{Accuracy} = \frac{284,315}{284,807} = 99.83\% \quad (3.2)$$

Ce classifieur détecterait zéro fraudes — absolument inutile pour notre objectif. L'accuracy ignore complètement la distribution des classes.

3.3.2 ROC-AUC (Area Under Curve)

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN} \quad (3.3)$$

L'AUC (Area Under the ROC Curve) mesure la probabilité qu'un exemple positif soit classé avec un score plus élevé qu'un exemple négatif. AUC est :

- **Insensible au déséquilibre** : Ne dépend pas de la distribution des classes
- **Indépendant du seuil** : Évalue la discrimination à tous les seuils
- **Interprétable** : Varie de 0 (classifieur parfait inverse) à 1 (classifieur parfait)
- **Standard industriel** : Largement utilisé en fraude bancaire

3.3.3 Précision et Rappel

$$\text{Précision} = \frac{TP}{TP + FP}, \quad \text{Rappel} = \frac{TP}{TP + FN} \quad (3.4)$$

- **Précision** : Proportion des fraudes détectées qui sont correctes (taux de fausses alarmes)
- **Rappel** : Proportion des vraies fraudes détectées (taux de détection)

Un bon système de détection de fraude équilibre les deux. Une précision élevée minimise les fausses alarmes (transactions légitimes refusées), tandis qu'un rappel élevé capture la majorité des vraies fraudes.

3.3.4 F1-Score

$$F1 = 2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (3.5)$$

Le F1-score est particulièrement utile pour les datasets déséquilibrés car il pénalise les cas où la précision et le rappel sont très différents.

Chapitre 4

Random Forest - Théorie et Implémentation

4.1 Concepts Fondamentaux

4.1.1 Arbres de Décision

Avantages :

- Faciles à interpréter
- Gèrent naturellement les non-linéarités
- Ne nécessitent pas de normalisation des features
- Peu de paramétrisation requise

Inconvénients :

- Sujets au surapprentissage (overfitting)
- Instables : petits changements dans les données → grands changements d'arbre
- Performance médiocre sur données simples

4.2 Ensemble Learning et Bagging

4.2.1 Motivation de l'Ensemble

Processus Bagging :

1. Créer B ensembles de bootstrap en ré-échantillonnant aléatoirement les données avec remplacement
2. Entraîner un arbre de décision indépendant sur chaque ensemble de bootstrap
3. Combiner les prédictions (moyenne pour régression, vote majoritaire pour classification)

Mathématiquement, si chaque arbre $h_b(x)$ a une erreur attendue $E[e]$, le Bagging réduit la variance à $\frac{E[e]}{B}$ en hypothèse d'indépendance.

4.3 Random Forest

4.3.1 Architecture et Algorithme

Algorithme Random Forest :

1. Pour chaque arbre $b = 1, \dots, B$:
 - (a) Créer un ensemble de bootstrap D_b en ré-échantillonnant les données
 - (b) Croître un arbre T_b sur D_b en :
 - i. Sélectionner aléatoirement $\sqrt{p}$ features à chaque split
 - ii. Choisir le split maximisant le gain d'information
 - iii. Laisser l'arbre croître sans limite (pas d'élagage)
2. Prédiction finale :
 - (a) Classification : $\hat{y} = \text{mode}(T_1(x), \dots, T_B(x))$
 - (b) Régression : $\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$

4.3.2 Importance des Features dans Random Forest

$$\text{Importance}(j) = \frac{1}{B} \sum_{b=1}^B \Delta \text{Impurity}_{j,b} \quad (4.1)$$

Cette mesure identifie les features les plus prédictives, facilitant l'interprétabilité du modèle.

4.4 Application à la Détection de Fraude

4.4.1 Hyperparamètres Configurés

TABLE 4.1 – Hyperparamètres Random Forest

Paramètre	Valeur	Justification
n_estimators	100	Équilibre compétence/coût computationnel
max_depth	None	Arbres libres d'explorer les patterns
criterion	'gini'	Indice de Gini pour splits
min_samples_split	2	Autoriser les divisions petites
random_state	2018	Reproductibilité

4.4.2 Performance Observée

Random Forest obtient un ROC-AUC de 0.953 sur l'ensemble de test. Bien que très bon, ce résultat est inférieur aux méthodes de boosting, en partie due à la non-pondération des exemples difficiles.

Chapitre 5

AdaBoost - Théorie et Implémentation

5.1 Principes du Boosting

5.1.1 Boosting vs Bagging

Intuition :

- Entraîner un premier classifieur faible sur tous les données
- Identifier les exemples mal classifiés
- Augmenter les poids de ces exemples difficiles
- Entraîner un nouveau classifieur qui se concentre sur ces exemples
- Répéter jusqu'à convergence

5.2 AdaBoost (Adaptive Boosting)

5.2.1 Algorithme Détaillé

1. Initialiser les poids : $w_i^{(1)} = \frac{1}{N}$ pour tous les exemples i
2. Pour chaque itération $t = 1, \dots, T$:
 - (a) Entraîner un classifieur faible h_t sur les données pondérées par $w^{(t)}$
 - (b) Calculer l'erreur pondérée :

$$\epsilon_t = \sum_{i=1}^N w_i^{(t)} \cdot \mathbf{1}(h_t(x_i) \neq y_i) \quad (5.1)$$

- (c) Si $\epsilon_t \geq 0.5$ ou $\epsilon_t = 0$: arrêter

(d) Calculer le poids du classifieur :

$$\alpha_t = \frac{1}{2} \log \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \quad (5.2)$$

(e) Mettre à jour les poids :

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(x_i)) \quad (5.3)$$

(f) Normaliser : $w^{(t+1)} \leftarrow \frac{w^{(t+1)}}{\sum_i w_i^{(t+1)}}$

3. Prédiction finale :

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right) \quad (5.4)$$

5.2.2 Interprétation Mathématique

$$L = \sum_{i=1}^N \exp(-y_i f(x_i)) \quad (5.5)$$

où $f(x) = \sum_t \alpha_t h_t(x)$ est le classifieur combiné. Cette fonction de perte exponentielle pénalise fortement les exemples mal classifiés, forçant le boosting à se concentrer sur les cas difficiles.

5.3 Application à la Détection de Fraude

5.3.1 Avantages pour notre Problème

- **Pondération des exemples** : Augmente naturellement les poids des minorités frauduleuses
- **Apprentissage séquentiel** : Chaque itération se concentre sur les cas précédemment mal classifiés
- **Pas de paramètre de régularisation explicite** : Le risque de surapprentissage est limité par le nombre d'itérations
- **Interprétabilité** : Les poids α_t indiquent la contribution de chaque classifieur

5.3.2 Configuration et Performance

AdaBoost obtient un ROC-AUC de 0.946, inférieur à Random Forest malgré ses avantages théoriques. Les algorithmes de boosting modernes (CatBoost, XGBoost, LightGBM) amélioreront considérablement cette performance.

Chapitre 6

Gradient Boosting et XGBoost

6.1 Théorie du Gradient Boosting

6.1.1 Perspective de l'Optimisation

Idée Principale :

Minimiser une fonction de perte en ajoutant itérativement des fonctions h_t qui pointent dans la direction du gradient négatif :

$$F_m(x) = F_{m-1}(x) + \gamma h_m(x) \quad (6.1)$$

où h_m approxime $-\nabla L(y, F_{m-1}(x))$, le gradient de la fonction de perte.

6.1.2 Algorithme Gradient Boosting Générique

1. Initialiser $F_0(x) = \arg \min_c \sum_i L(y_i, c)$ (une constante)
2. Pour chaque itération $m = 1, \dots, M$:
 - (a) Calculer les pseudo-résidus (gradient négatif) :

$$r_{i,m} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)} \quad (6.2)$$

- (b) Entraîner un arbre de régression h_m pour prédire $r_{i,m}$ à partir de x_i
 - (c) Trouver le multiplicateur optimal :

$$\gamma_m = \arg \min_{\gamma} \sum_i L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)) \quad (6.3)$$

- (d) Mettre à jour :

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x) \quad (6.4)$$

Cette approche est remarquablement générale et s'applique à n'importe quelle fonction de perte différentiable.

6.2 XGBoost (Extreme Gradient Boosting)

6.2.1 Innovations de XGBoost

1. **Régularisation explicite** : Ajoute des termes de régularisation L1 et L2

$$L_{obj} = \sum_i L(y_i, \hat{y}_i) + \sum_t \Omega(f_t) \quad (6.5)$$

où $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda w^2$ (nombre de feuilles + norme des poids)

2. **Approximation du second ordre** : Utilise l'expansion de Taylor au second ordre

$$L \approx \sum_i \left[L(y_i, F^{(t-1)}) + g_i f_t + \frac{1}{2} h_i f_t^2 \right] + \Omega(f_t) \quad (6.6)$$

où g_i et h_i sont les gradients first et second

3. **Scission de features optimisée** : Utilise un algorithme rapide de scission linéaire (weighted quantile sketch)
4. **Gestion du déséquilibre** : Paramètre `scale_pos_weight` pondère les exemples positifs

6.2.2 Avantages de XGBoost

- **Performance** : État de l'art sur de nombreux problèmes (kaggle, compétitions)
- **Régularisation** : Contrôle explicite de la complexité du modèle
- **Vitesse** : Optimisations computationnelles permettent scalabilité
- **Flexibilité** : Nombreuses fonctions de perte supportées
- **Feature Importance** : Calculs automatiques des contributions des features

6.3 Configuration et Application

6.3.1 Hyperparamètres XGBoost

XGBoost obtient un ROC-AUC de 0.973, une amélioration significative par rapport aux méthodes précédentes.

Chapitre 7

CatBoost - Théorie et Implémentation

7.1 Spécificités de CatBoost

7.1.1 Motivation pour les Features Catégorielles

1. **Gestion native des catégories** : Pas d'encoding one-hot nécessaire
2. **Boosting ordonné (Ordered Boosting)** : Réduit le surapprentissage
3. **Arbres obélisques (Oblivious Trees)** : Arbres symétriques plus stables
4. **Optimisations GPU** : Support de l'accélération GPU

7.2 Ordered Boosting

7.2.1 Problème du Boosting Traditionnel

CatBoost résout ce problème avec l'Ordered Boosting :

1. Créer une permutation aléatoire des exemples : $\sigma = (\sigma(1), \dots, \sigma(N))$
2. Pour chaque arbre m :
 - (a) Pour chaque exemple i dans l'ordre de la permutation :
 - i. Utiliser le modèle entraîné sur les exemples précédents pour prédire $\hat{y}_i$
 - ii. Calculer le résidu : $r_i = y_i - \hat{y}_i$
 - iii. Ajouter cet exemple au training set pour les prochains gradients

Cela garantit que le gradient pour chaque exemple est indépendant de cet exemple lui-même.

7.3 Arbres Obélisques

7.3.1 Définition et Avantages

- Sont plus stables numériquement
- Réduisent le surapprentissage
- Sont plus faciles à paralléliser
- Donnent des intervalles de confiance théoriques plus étroits

7.4 Configuration CatBoost pour Notre Problème

7.4.1 Hyperparamètres

CatBoost obtient un ROC-AUC de 0.970, une performance excellente, très proche de XGBoost.

Chapitre 8

LightGBM - Théorie et Implémentation

8.1 Innovation Clé : Leaf-wise Tree Growth

8.1.1 Level-wise vs Leaf-wise

LightGBM utilise la croissance **leaf-wise** : à chaque itération, on élargit la feuille qui minimise le plus la perte (meilleur-d'abord). Cela :

- **Atteint une meilleure convergence** : Avec le même nombre d'itérations
- **Crée des arbres plus profonds** : Structure plus diverse
- **Risque de surapprentissage** : Nécessite plus de régularisation

Illustration :

- Level-wise : Élargir le niveau k complètement avant le niveau $k + 1$
- Leaf-wise : Choisir la feuille avec le meilleur gain, la diviser, répéter

8.1.2 Comparaison Mathématique

Pour le même nombre de feuilles L , LightGBM obtient une perte training inférieure, justifiant la meilleure performance empirique.

8.2 Gradient-based One-Side Sampling (GOSS)

8.2.1 Réduction de Données Sans Perte

1. Calculer le gradient absolu pour chaque exemple : $|g_i|$
2. Trier les exemples par gradient (décroissant)
3. Garder les top $a\%$ d'exemples avec les plus grands gradients (plus informatifs)
4. Garder aléatoirement $b\%$ des exemples restants

5. Ré-pondérer les exemples gardés aléatoirement par $\frac{1-a}{b}$

Cette stratégie conserve les exemples difficiles (grands gradients) tout en sous-échantillonnant les exemples faciles, réduisant significativement le temps d'entraînement sans compromettre la performance.

8.3 Histogram-based Learning

8.3.1 Groupage des Features Continues

1. Pour chaque feature continue, créer n_{bins} bins (généralement 255)
2. Assigner chaque valeur au bin le plus proche
3. Calculer l'histogramme (nombre d'exemples par bin)
4. Utiliser l'histogramme pour trouver les meilleurs splits

Avantages :

- Réduit la mémoire : $O(\text{données}) \rightarrow O(\text{features} \times \text{bins})$
- Accélère les computations : opérations sur histogrammes plus rapides
- Peu de perte d'information : bins fins captent les patterns
- Régularisation implicite : quantification = smoothing

8.4 Configuration et Performance

8.4.1 Hyperparamètres Optimisés

Performance : LightGBM obtient un **ROC-AUC de 0.976**, le meilleur de tous les modèles évalués, avec une précision de 91.9% et un rappel de 88.0%.

8.4.2 Justification du Succès

1. **Leaf-wise growth** : Optimise précisément sur la métrique de perte
2. **GOSS** : Concentre l'apprentissage sur les exemples importants (fraudes)
3. **Efficiency** : Permet plus d'itérations dans le même temps
4. **Regularization** : Les paramètres de `min_data_in_leaf` et `num_leaves` contrôlent bien la complexité

Chapitre 9

Résultats et Comparaisons Expérimentales

9.1 Performance Globale

9.1.1 ROC-AUC Scores

Observations :

- Tous les modèles dépassent largement le baseline (0.5 = classifieur aléatoire)
- Les méthodes de boosting surpassent Random Forest
- LightGBM présente le meilleur ROC-AUC et le plus petit écart train-test

9.1.2 Métriques Détaillées

Interprétation :

- **Précision 91.9% (LightGBM)** : Sur 100 fraudes détectées, 92 sont vraies. Taux de fausse alarme très acceptable
- **Rappel 88.0% (LightGBM)** : Le modèle détecte 88% des vraies fraudes. Seulement 12% sont manquées
- **F1-Score 0.899** : Équilibre excellent entre précision et rappel

9.2 Analyse de la Généralisabilité

9.2.1 Écart Train-Test

Analyse :

- LightGBM a l'écart le plus petit (2.1%), indiquant la meilleure généralisation
- Tous les écarts sont modérés, pas d'overfitting sévère

- Les algorithmes de boosting modernes (CatBoost, XGBoost, LightGBM) gèrent mieux la généralisation

9.3 Importance des Features

9.3.1 Top Features selon LightGBM

Insights :

- V14, V12, V10 combinées représentent 37.5% de l'importance
- Ces trois composantes PCA captent les patterns de fraude majeurs
- Amount (importance 5.5%) est modérément important
- Time est peu important, suggérant que la fraude n'est pas fortement concentrée temporellement

Chapitre 10

Conclusion et Perspectives Futures

10.1 Résumé des Findings

Conclusions clés :

1. **LightGBM est le meilleur modèle** avec un ROC-AUC de 0.976, une précision de 91.9% et un rappel de 88.0%
2. **Les méthodes de boosting surpassent le bagging** : Les algos de boosting (CatBoost, XGBoost, LightGBM) obtiennent des performances supérieures à Random Forest et AdaBoost
3. **Gestion efficace du déséquilibre** : Tous les modèles gèrent bien le déséquilibre extrême (1 :579) via les métriques appropriées et les algorithmes intelligents
4. **Bonne généralisation** : L'écart train-test est faible (2.1
5. **Interprétabilité** : Les analyses d'importance des features révèlent que V14, V12, V10 sont les prédicteurs dominants de fraude

10.2 Recommandations Pratiques

10.2.1 Pour le Déploiement en Production

Monitoring continu : Suivre la dérive de modèle (concept drift) et les changements distributionnels

Réentraînement régulier : Actualiser le modèle mensuellement ou trimestriellement avec de nouvelles données

Threshold tuning : Ajuster le seuil de décision basé sur les coûts métier (fausse alerte vs fraude manquée)

Explainability : Implémenter SHAP ou LIME pour expliquer les décisions aux auditeurs

10.3 Perspectives Futures

10.3.1 Améliorations Possibles

Deep Learning : Explorer les réseaux de neurones (autoencodeurs, LSTM) pour patterns séquentiels

Sampling Avancé : Appliquer SMOTE, ADASYN ou autres techniques d'oversampling

Feature Engineering : Créer des features temporelles et comportementales supplémentaires

Cost-Sensitive Learning : Pondérer explicitement les coûts asymétriques (faux positif vs faux négatif)

Détection Anomale : Utiliser des méthodes non-supervisées (Isolation Forest, One-Class SVM) pour les fraudes nouvelles

10.4 Impact Potentiel

- **Prévenir des millions de fraudes annuelles** dans une grande institution bancaire
- **Économiser des milliards de dollars** en pertes évitées
- **Améliorer l'expérience client** en réduisant les fausses alertes
- **Renforcer la confiance** dans les systèmes de paiement numérique

10.5 Conclusion Finale

LightGBM émerge comme la meilleure solution globale, offrant un excellent équilibre entre performance, efficacité et interprétabilité. Sa supériorité découle de innovations importantes comme la croissance leaf-wise, GOSS, et les histogrammes, qui visent à résoudre spécifiquement les défis rencontrés dans les données réelles.

En conclusion, l'apprentissage automatique modernes, appliqué correctement avec une compréhension théorique profonde, peut transformer les capacités de détection de fraude des institutions financières et contribuer à un écosystème de paiement plus sûr pour tous.

Bibliographie

- [1] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
- [2] Freund, Y., Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119-139.
- [3] Chen, T., Guestrin, C. (2016). XGBoost : A scalable tree boosting system. In *Proceedings of the 22nd KDD International Conference*, 785-794.
- [4] Ke, G., et al. (2017). LightGBM : A fast, distributed, high-performance gradient boosting framework. In *Advances in Neural Information Processing Systems*, 3149-3157.
- [5] Dorogush, A. V., Ershov, V., Gulin, A. (2018). CatBoost : gradient boosting with categorical features support. In *Workshop on Machine Learning Systems at NeurIPS 2018*.
- [6] Pozzolo, A. D., et al. (2014). Calibrating probability with undersampling for unbalanced classification. In *Computational Intelligence and Data Mining (CIDM)*, 159-166.
- [7] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874.
- [8] Friedman, J. H. (2001). Greedy function approximation : a gradient boosting machine. *Annals of Statistics*, 29(5), 1189-1232.